Pre-Lab Tasks – Graph

1. Implement Adjacency list representation of Graph. Provide all the necessary functions stated below to complete the ADT. Also, make the ADT templated.

    - Create an empty graph
    - Destroy a graph
    - isEmpty
    - Determine number of vertices in a graph
    - Determine number of edges in a graph
    - Determine whether an edge exists between two vertices
    - Insert a vertex in a graph
    - Insert an edge between two vertices
    - Delete a vertex from graph
    - Delete an edge between two vertices
    - Search a vertex from graph
    - Breadth First Search
    - Depth First Search
    - Find shortest path between two vertices.
    - Compute and return the Minimum Spanning Tree of the graph

2. Design and implement an **Airline Route Optimization System** using a **directed, weighted Graph ADT (Adjacency List, templated in C++ from task 1)**. The system will model airports and flight routes while enabling computation of optimal travel paths.

    In this scenario:

    - **Vertices** represent airports.
    - **Directed edges** represent flights between airports.
    - **Weights** represent ticket cost.

    The system should allow management of the flight network and computation of the **shortest path between two airports**.

2. Functional Requirements

- **Graph Operations**
    - Add airports (vertices)
    - Remove an airport and all associated flights.
    - Add/remove directed flights (edges with weights)
    - Check if a flight exists between two airports
    - Display total vertices and edges
    - Find:
        - All outgoing flights from a given airport.
        - All incoming flights to a given airport.
- **Traversal**
    - Perform **BFS** to explore reachable airports
    - Perform **DFS** to analyze network structure
- **Shortest Path**
    - Compute shortest path between a source and destination
    - Display:
        - Optimal route
        - Total cost